

Menú online + carrito con pedido por WhatsApp

Arquitectura y código base de un menú online 100% estático para locales gastronómicos: sin base de datos tradicional, sin backend, sin pasarela de pago y sin publicar datos bancarios. El panel de administración es un documento de Google Sheets y el pedido se cierra por WhatsApp.

Dato	Valor
Stack	React 19 + Vite 8 + Tailwind CSS 4 + PapaParse
Base de datos	Google Sheets publicado como CSV (solo lectura)
Administración	El propio Google Sheet: alta, edición, baja y stock
Pagos	Efectivo o transferencia. La web no publica alias ni CBU: el local lo pasa por el chat
Confirmación	wa.me/número?text=... con el pedido estructurado
Demo online	https://catalogo-whatsapp-nexa-kofi.vercel.app/ (Vercel, cabeceras activas)
Hosting	Vercel / Netlify / Cloudflare Pages / GitHub Pages (plan gratuito)
Costo mensual del local	Dominio (ej. .com.ar) — sin servidores ni licencias
Build de producción	306,3 kB JS (95,1 kB gzip) + 37,6 kB CSS (7,8 kB gzip)
Fecha del documento	23/09/2026

Idea central. El navegador del cliente hace un *fetch* al CSV del Google Sheets, arma el pedido en el carrito y, al confirmar, abre WhatsApp con el mensaje ya escrito. No hay servidor que mantener, no hay datos de tarjetas ni datos bancarios en juego, y no existe superficie para inyección SQL porque no hay SQL.

1. Resumen del sistema

El sistema tiene dos pantallas de uso y una planilla:

- **Vista cliente:** menú con buscador y filtros, carrito lateral (off-canvas), checkout en dos pantallas (datos y entrega, forma de pago) y envío del pedido por WhatsApp.
- **Panel administrador:** el Google Sheets. El dueño del local carga, edita, oculta o borra productos y cambia el stock sin tocar código. Los cambios se ven en la web al recargar la página.
- **Sin backend ni base de datos:** el sitio es un paquete de archivos estáticos. No hay API keys, no hay credenciales, no hay costos de infraestructura.
- **Sin datos de cobro en la web:** no hay pasarela, ni formulario de tarjeta, ni alias ni CBU publicados. Si el cliente paga por transferencia, el alias viaja por el chat.

Flujo completo

CLIENTE		DUEÑO DEL LOCAL
-----		-----
1. abre la web y ve el MENU	<-- CSV <--	Google Sheets (carga el menu)

```

    (los productos van primero)                (agrega, edita, oculta, stock)
2. elige productos
3. elige entrega: envío a domicilio o retiro
4. elige pago: efectivo o transferencia
  (no se muestran alias ni CBU)
5. "Enviar pedido" ==> wa.me con el mensaje ==> WhatsApp del local
  (n de pedido, detalle, totales,             ==> prepara el pedido
   entrega, pago y datos del cliente)         ==> si es transferencia,
                                              pasa el alias por el chat

El pago NUNCA pasa por la web y la web NO publica datos bancarios:
se coordina por whatsapp (alias a pedido, o efectivo al recibir).
En el celular, una barra fija mantiene el pedido a un toque.

```

Por qué este diseño conviene al local

- **Costo casi nulo:** hosting gratuito + dominio. Nada de servidores, bases de datos ni suscripciones de pasarelas.
- **Cero riesgo financiero:** al no integrar cobros, no hay comisiones, ni cumplimiento PCI-DSS, ni datos de tarjetas, ni datos bancarios publicados.
- **Menos fricción para el cliente:** no hay registro, no hay pantallas intermedias de presentación y el pedido sale desde el carrito en dos pasos.
- **Administración que el dueño ya sabe usar:** agrega y edita productos en una planilla (incluso desde el celular), sin panel nuevo ni contraseñas.
- **El pedido llega completo a WhatsApp:** número de pedido, detalle, totales, entrega, forma de pago y datos del cliente. Menos ida y vuelta, menos pedidos mal tomados.
- **Publicación inmediata:** es un sitio estático: se copia a cualquier hosting y funciona.

2. El Google Sheet: base de datos y panel de administración

Una sola pestaña, con la fila 1 como encabezado. El nombre de las columnas es tolerante: no distingue mayúsculas ni tildes y acepta sinónimos (por ejemplo *título*, *titulo*, *nombre* o *producto*; *imagen*, *foto* o *url_imagen*).

Cómo se administra el menú (no hay otro panel que el Sheet)

Quiero...	Qué hago en la planilla	Qué pasa en la web
Agregar un producto	Escribo una fila nueva con id , título y precio (el resto es opcional)	Aparece en el menú al recargar (caché de 10 min)
Modificar precio o texto	Edito la celda	El cliente lo ve al recargar la página
Eliminar un producto	Opción limpia y reversible: activo = no . Opción final: borro la fila	Deja de mostrarse (y sale del carrito guardado)
Cambiar el stock	stock = cantidad	Con 0 se muestra "Agotado" y no se puede pedir
Poner una oferta	Escribo el precio rebajado en precio_oferta	Precio anterior tachado + porcentaje de descuento
Reordenar el menú	orden = 1, 2, 3... y destacado = si	Los destacados van primero y después manda el orden
Crear o quitar categorías	Uso o dejo de usar palabras en categoría	Los filtros de arriba se arman solos desde la planilla
Agregar un filtro (vegano, sin TACC...)	Escribo esa palabra en etiquetas	Aparece como chip en "Filtros rápidos": no hay lista fija
Cargar ingredientes	Escribo el texto en ingredientes	Se muestra en la ficha del producto, con "Ver ingredientes" en la tarjeta

¿El stock se descuenta solo?

- **No, y es una decisión de arquitectura.** Descontar stock exige un lugar donde *escribir*: un servidor o una base de datos. Acá la planilla se lee y nunca se modifica desde la web.
- **Cómo se maneja en la práctica:** cuando algo se termina, el dueño pone `stock = 0` desde el celular y en la web aparece "Agotado" (no se puede pedir). Son segundos y lo decide quien está en la cocina.
- **Si algún día lo quieren automático:** se agrega un Google Apps Script como mini API gratuita que descuenta al entrar cada pedido. Único camino sin pagar servidores; el riesgo a conversar es el de dos pedidos simultáneos del último producto.
- **Lo que sí hace la web:** avisar. El cliente ve "Agotado" y el pedido que llega al local lleva el detalle para confirmar disponibilidad.

Cómo se publican las fotos de los productos

- La planilla guarda **un link público**, no el archivo. **Google Drive ya está integrado:** se comparte la foto como "cualquiera con el enlace" y la web convierte sola el link de compartir en link directo (`/file/d/ID/view` → `uc?export=view&id=ID`).
- **Respaldo automático:** si ese primer link falla, la web prueba el otro formato de Drive (`lh3.googleusercontent.com/d/ID`) antes de dar la imagen por rota. Verificado con una prueba en navegador: intenta el primero, intenta el segundo y muestra la foto.
- **Respaldo a medida:** en la celda se pueden escribir dos links separados por "|" (`link1|link2`) y la web usa el segundo si el primero falla.

- Lo más estable: subir las fotos al hosting del sitio o a un servicio de imágenes (Imgur, Cloudinary): no dependen de permisos de Drive.
- Si todo falla, la tarjeta muestra la inicial del producto en lugar del ícono de imagen rota: la grilla nunca se ve “rota”.
- **Qué no publicar:** la hoja publicada como CSV es pública (incluye productos con activo = no y la columna stock). Costos, márgenes y notas internas van en otra pestaña que no se publica.

Se puede editar desde el celular con la app de Google Sheets. No hay usuarios, contraseñas ni panel que mantener: el dueño necesita una cuenta de Google (gratis) y publicar la hoja como CSV una sola vez; la URL no cambia nunca, así que no hay que volver a publicar el sitio.

Si algún día piden un panel web con login

- **Google Apps Script** como API (sigue siendo gratis): una Web App lee y escribe en la misma hoja, protegida con una clave, y la web suma una pantalla de administración. Es la opción más barata para no dejar de ser “cero servidor”.
- **Backend real** (Supabase / Firebase): panel cómodo y multiusuario, pero agrega costos, cuentas y mantenimiento. Se justifica con varios locales, stock que se descuenta solo o pedidos guardados en base de datos.

Columna	Oblig.	Ejemplo	Qué hace / formato
id	no	MIL01	Identifica el producto. Si falta se genera desde el título. Debe ser único: el carrito agrupa por id.
título	SÍ	Milanesa napolitana	Nombre visible. Una fila sin título se ignora.
descripcion	no	La clásica, al molde.	Texto libre. En la tarjeta se recorta a 2 líneas.
ingredientes	no	Masa, salsa, muzzarella...	Se muestra en la ficha del producto (foto grande + detalle).
precio	SÍ	9800	Acepta 9800, 9.800,50, \$ 9.800. La coma decimal se interpreta sola.
precio_oferta	no	8500	Si es menor al precio, se muestra tachado y con el % de descuento.
categoria	no	Rotisería	Con esto se arman los filtros de arriba del catálogo.
url_imagen	no	https://...jpg	Link público de la foto. En Drive: pegar el link de compartir y la web lo convierte. Se puede agregar un respaldo con “ ”: link1 link2
stock	no	12	0 = Agotado (no se puede agregar). Vacío = sin control de stock.
unidad	no	kg	Se muestra como “/ kg” junto al precio.
etiquetas	no	Vegano, Más pedida	Hasta 2 visibles en la tarjeta y todas se vuelven filtros rápidos.
destacado	no	si	Los destacados se ordenan primero en el catálogo.
activo	no	si	Con “no” el producto no se publica (borrado lógico, reversible).
orden	no	1	Orden manual dentro de la categoría (1, 2, 3...).

Cómo publicarla para que la web la lea

- **Opción A (recomendada).** Archivo → Compartir → **Publicar en la web** → elegir la pestaña y el formato **CSV** → Publicar → copiar la URL y pegarla en catalogo.hojaCsv. La hoja queda de solo lectura pública.
- **Opción B.** Compartir como “cualquiera con el enlace puede ver” y pegar solo el ID del documento en catalogo.hojaId más el gid de la pestaña: la app arma el endpoint CSV de Google.

- **Validación previa.** El proyecto incluye un validador que usa el mismo parser que la web: `npm run validar <archivo.csv o URL>`. Avisa de filas sin precio, imágenes faltantes, ids repetidos y errores de comillas.

Reglas de datos que conviene respetar

- No repetir **id**: dos filas con el mismo id se comportan como un mismo producto en el carrito.
- Si un texto tiene **comas** (por ejemplo “Docena con membrillo, crema y dulce”), la celda debe ir entre comillas cuando se exporta a CSV — el validador lo detecta.
- Evitar cambiar el **id** de un producto existente: se pierde el historial del carrito guardado.
- Los precios son los que se muestran y con los que se calcula el total; la web nunca los inventa.

3. Servicio de obtención de datos (Sheets → JSON)

Vive en `src/services/catalogo.js`. No importa React: es una función pura sobre texto CSV y un *fetch*. Eso permite reusarla desde un script de Node (el validador usa exactamente esta función).

```
// 1) Origen de datos (src/config/negocio.js)
export function urlHojaCsv() {
  const { hojaCsv, hojaId, gid, modoDemo } = NEGOCIO.catalogo
  if (hojaCsv?.trim()) return hojaCsv.trim() // CSV publicado
  if (hojaId?.trim()) { // endpoint gviz
    return `https://docs.google.com/spreadsheets/d/${hojaId.trim()}/gviz/tq?tqx=out:csv&gid=${gid}`
  }
  return modoDemo ? `${import.meta.env?.BASE_URL ?? ''}productos-demo.csv` : ''
}

// 2) Descarga + normalización + caché (fragmento)
export async function obtenerCatalogo({ forzar = false } = {}) {
  const url = urlHojaCsv()
  if (!url) throw new Error('Falta configurar el Google Sheet en negocio.js')
  const cache = leerCache() // localStorage 'catalogo.cache.v1'
  if (cache && !forzar && !cacheVencido(cache)) return { ...cache, desdeCache: true }

  let res
  try { res = await fetch(url, { redirect: 'follow' }) }
  catch (e) { return cache ? { ...cache, desdeCache: true, avisoRed: true } : Promise.reject(e) }
  if (!res.ok) throw new Error(`El catálogo respondió ${res.status}`)

  const texto = await res.text()
  // Si la hoja no es pública, Google devuelve el HTML del login:
  if (/^<!doctype|html/i.test(texto)) throw new Error('La hoja no está publicada como CSV.')

  const datos = transformarCatalogo(texto)
  guardarCache({ ...datos, actualizado: new Date().toISOString(), guardadoEn: Date.now() })
  return { ...datos, desdeCache: false }
}
```

Normalización de cada fila (PapaParse → objeto de producto)

```
const { data } = Papa.parse(csv.trim(), {
  header: true, // primera fila = nombres de columna
  skipEmptyLines: 'greedy', // ignora las filas fantasma que agrega Sheets
  transformHeader: h => normalizarTexto(h).replace(/s+/g, '_'),
})

// Alias aceptados por columna: el dueño escribe como quiere
const ALIAS = {
  titulo: ['titulo', 'título', 'nombre', 'producto', 'artículo'],
  precio: ['precio', 'precio_lista', 'valor'],
  oferta: ['precio_oferta', 'oferta', 'precio_promo', 'precio_promocional'],
  imagen: ['url_imagen', 'imagen', 'foto', 'url_foto', 'url'],
  activo: ['activo', 'visible', 'publicado'],
  // ... categoria, descripcion, stock, unidad, etiquetas, destacado, orden
}

// Fila cruda -> producto canónico que consume React
{
  id, titulo, descripcion,
  precio, precioOferta, // número o null
  precioFinal, // oferta si existe y es menor
  categoria, urlImagen, unidad, etiquetas,
  stock, sinStock: stock === 0, // -1 = sin control de stock
  destacado, activo, orden,
}
```

Decisiones que hacen que esto no se rompa

- **Caché en localStorage con vencimiento (10 min por defecto):** la web abre al instante con el último catálogo guardado y refresca en segundo plano (stale-while-revalidate).
- **Si Google falla, la tienda sigue vendiendo:** se muestra el catálogo en caché y un aviso discreto. Nunca una pantalla en blanco.
- **Errores explicados para el humano:** “La hoja no está pública. Archivo > Compartir > Publicar en la web > CSV.”
- **Avisos no bloqueantes:** los productos sin precio válido se listan en un desplegable para que el dueño los corrija, sin romper el catálogo.
- **Parseo de precios tolerante:** 1.500,50 / \$1500 / 1,500.50 se interpretan correctamente; una oferta mayor al precio se descarta para no inflar el total.

4. Lógica del carrito y generación del mensaje de WhatsApp

Estado global con reducer (reglas del carrito en un solo lugar)

```
// src/state/carritoReducer.js — acciones: AGREGAR · CAMBIAR_CANTIDAD · QUITAR
// NOTA · VACIAR · SET_ENTREGA · SET_PAGO · SET_DATOS · SINCRONIZAR · HIDRATAR
case 'AGREGAR': {
  const existente = estado.items.find(i => i.id === accion.producto.id)
  if (existente) {
    // mismo producto: suma cantidad
    return { ...estado, items: estado.items.map(i => i.id === accion.producto.id
      ? { ...i, cantidad: Math.min(i.cantidad + (accion.cantidad || 1), MAX_POR_LINEA) } : i) }
  }
  return { ...estado, items: [...estado.items, lineaDesdeProducto(accion.producto,
    accion.cantidad || 1)] }
}

// SINCRONIZAR: cuando llega un catálogo nuevo, refresca precios y stock de las
// líneas ya cargadas (y elimina lo que se dio de baja) sin perder cantidades.
```

Totales, envío y validación (funciones puras, testeadas)

```
// src/services/pedido.js
export function calcularTotales(items, entrega = 'envio') {
  const lineas = items.map(item => ({
    ...item,
    precioUnitario: item.precioFinal ?? item.precio,
    subtotalLinea: (item.precioFinal ?? item.precio) * item.cantidad,
  }))
  const subtotal = lineas.reduce((a, l) => a + l.subtotalLinea, 0)
  const unidades = lineas.reduce((a, l) => a + l.cantidad, 0)
  const usaEnvio = entrega === 'envio' && NEGOCIO.entrega.envio.habilitado
  const envio = usaEnvio && !envioGratis(subtotal) ? NEGOCIO.entrega.envio.costos : 0
  return { lineas, subtotal, unidades, envio, total: subtotal + envio }
}

export function validarCheckout({ items, entrega, pago, datos }) {
  const errores = {}
  if (!items.length) errores.items = 'El carrito está vacío.'
  if (datos.nombre.trim().length < 3) errores.nombre = 'Ingresa tu nombre y apellido.'
  if (datos.telefono.replace(/\D/g, '').length < 8) errores.telefono = 'Teléfono inválido.'
  if (entrega === 'envio' && datos.direccion.trim().length < 5)
    errores.direccion = 'Ingresa la calle, número y localidad.'
  return { valido: Object.keys(errores).length === 0, errores } // la UI marca cada campo
}

export function generarNumeroOrden(prefijo = 'PED') { // PED-261004-7K3F
  // fecha (aammdd) + 4 caracteres de un alfabeto sin letras/números ambiguos
}
```

Generador del mensaje y del link (src/services/whatsapp.js)

```
export function construirMensajePedido(pedido, negocio = NEGOCIO) {
  const L = []
  L.push(`*NUEVO PEDIDO ${pedido.numeroOrden}*`) // *negrita* = marcado de WhatsApp
  L.push(`${negocio.marca.nombre} — ${fechaLegible(pedido.fecha)}`)
  L.push('', '*Detalle*')
  pedido.items.forEach(i => {
    L.push(`• ${i.cantidad} x ${i.titulo}${i.unidad ? ` (${i.unidad})` : ''} — ${formatearPrecio(i.subtotalLinea)}`)
    if (i.nota) L.push(`■ ${i.nota}`) // "sin cebolla", etc.
  })
  L.push('', `Subtotal: ${formatearPrecio(pedido.subtotal)}`)
  if (pedido.entrega === 'envio') {
    L.push(pedido.envio > 0 ? `Envío: ${formatearPrecio(pedido.envio)}` : 'Envío: sin cargo')
  }
}
```



```

    L.push(`*TOTAL: ${formatearPrecio(pedido.total)}*`)
    L.push('', '*Entrega*', pedido.entregaTexto)
    if (pedido.entrega === 'envio') L.push(`Dirección: ${pedido.cliente.direccion}`)
    else
        L.push(`Retiro en: ${negocio.entrega.retiro.direccion}`)
    // La web no publica datos bancarios: la línea del mensaje avisa al local
    // si tiene que pasar el alias (transferencia) o si se abona al recibir.
    const pago = pedido.pago === 'transferencia' ? negocio.pago.transferencia : negocio.pago.efectivo
    L.push('', '*Pago*', pago.lineaMensaje)
    L.push('', '*Cliente*', pedido.cliente.nombre, `Teléfono: ${pedido.cliente.telefono}`)
    return L.join('\n')
  }
  export function construirUrlWhatsapp(mensaje, numero = NEGOCIO.whatsapp.numero) {
    return `https://wa.me/${limpiarTelefono(numero)}?text=${encodeURIComponent(mensaje)}`
  }
}

```

Mensaje real generado durante la verificación

```

*NUEVO PEDIDO – Valentin Jurado*
Pizzería – 22/09/2026 23:40

*Detalle*
• 1 x Pizza muzzarella al molde (8 porciones) – $ 9.800,00
• 2 x Hamburguesa doble cheddar (unidad) – $ 19.800,00

Subtotal: $ 29.600,00
Envío: sin cargo (promoción)
Ahorro por ofertas: -$ 2.000,00
*TOTAL: $ 29.600,00*

*Entrega*
Envío a domicilio
Dirección: Rivadavia 1234, Tandil
Horario: entre 20 y 21

*Pago*
Transferencia: pasame el alias para transferir.

*Cliente*
Valentin Jurado
Teléfono: 2494 123456

```

El mensaje no incluye ningún dato bancario: la línea de pago se arma desde la configuración (pago.transferencia.lineaMensaje) y, cuando el cliente elige transferencia, el propio pedido le avisa al local que tiene que pasar el alias. Si elige efectivo, la línea dice que se abona al recibir.

Identificación: el pedido se reconoce por el **nombre y apellido** del cliente, que viaja en la primera línea (así el local lo lee en la notificación sin abrir el chat). El número de pedido es opcional y viene apagado: si un local lo pide, se activa con `pedido.mostrarNumeroOrden: true` y aparece como una línea más, sin tocar el resto del flujo.

Pantallas del flujo de pago (sólo se elige cómo pagar)

- **El menú es lo primero:** buscador, filtros por categoría y **filtros rápidos** que salen de la columna *etiquetas* (Vegetariano, Vegano, Sin TACC, Picante...). Sin pantallas de presentación de por medio.
- **Ficha del producto:** tocando la foto o el título se abre el detalle con foto grande, descripción completa, **ingredientes**, etiquetas y selector de cantidad. Resuelve el “¿qué lleva?” sin llamar al local.
- **Pedido lateral (off-canvas):** cantidades, aclaración por producto, modalidad de entrega, barra de progreso hacia el envío gratis y totales.
- **Pantalla 1 – Datos y entrega:** nombre, teléfono, dirección (sólo si elige envío), referencia para el repartidor, horario preferido y aclaraciones. Validación por campo.
- **Pantalla 2 – Forma de pago:** Efectivo o Transferencia, y el botón verde *Enviar pedido* que abre WhatsApp con todo escrito. No hay pantalla de confirmación aparte, ni datos bancarios, ni campos de tarjeta.

- **Cierre:** número de pedido a la vista, botón para reabrir WhatsApp y opción de copiar el pedido como comprobante del armado.

El cliente no deja rastro en su dispositivo

La web no usa `localStorage`, `cookies` ni `sessionStorage`: no guarda el pedido, ni los datos personales, ni el menú en el teléfono de quien compra. Todo vive en la memoria de la página (y el menú se cachea en memoria durante la visita para no golpear a Google en cada interacción). Si el cliente recarga, el pedido arranca vacío. Está verificado en el E2E (*no guarda nada en el dispositivo*).

5. Estructura del proyecto

```

catalogo-whatsapp/
├── public/
│   ├── productos-demo.csv          catálogo de ejemplo (modo demostración)
│   ├── favicon.svg
│   └── docs/
│       ├── plantilla-google-sheet.csv  plantilla lista para importar en Sheets
│       ├── ARQUITECTURA.pdf          este documento
│       ├── generar-pdf-arquitectura.py  generador de este PDF
│       └── scripts/
│           ├── validar-catalogo.mjs    valida tu CSV con el mismo parser de la web
│           ├── test-servicios.mjs      21 pruebas de la lógica pura (sin navegador)
│           ├── cdp.mjs                 conexión CDP reutilizable (capturas y E2E)
│           ├── e2e.mjs                 flujo de compra completo: 45 comprobaciones
│           ├── auditar-privacidad.mjs  auditoría de red, datos del cliente y cabeceras
│           ├── capturar.mjs            capturas de pantalla para la ficha de venta
│           ├── vercel.json              cabeceras de seguridad (Vercel)
│           ├── public/_headers          cabeceras de seguridad (Netlify / Cloudflare Pages)
│           └── src/
│               ├── config/
│               │   ├── negocio.js      ■■■ ÚNICO archivo que se edita por cliente
│               │   ├── paletas.js      6 paletas de color aplicadas en runtime (CSS vars)
│               │   └── services/        lógica sin React, testeable
│               │       ├── catalogo.js  CSV de Sheets → JSON (+ caché + errores claros)
│               │       ├── pedido.js    totales, envío, validaciones
│               │       ├── whatsapp.js  mensaje estructurado + link wa.me
│               │       └── formato.js   precios, fechas, normalización de texto
│               ├── state/
│               │   ├── carritoReducer.js  reglas del carrito (puro)
│               │   └── CarritoContext.jsx  estado global + persistencia en localStorage
│               ├── hooks/
│               │   ├── useCatalogo.js    carga con caché y refresco en segundo plano
│               │   └── useCopiar.js       copiar (pedido) con respaldo y aviso de fallo
│               ├── components/
│               │   ├── layout/           Header · Footer
│               │   ├── catalogo/         Catalogo · Filtros · ProductCard · FichaProducto
│               │   ├── carrito/          CartWidget (off-canvas) · ItemCarrito · SelectorEntrega · BarraPedidoMovil
│               │   ├── checkout/         CheckoutModal · PasoDatos · PasoPago · PantallaExito
│               │   └── ui/               Boton · Modal · Toast · Campo · ImagenProducto · Estados · Iconos
│               ├── App.jsx               composición de la página
│               └── main.jsx               tema, SEO y providers

```

Dos reglas de oro para que sea mantenible y vendible

- **services/ nunca importa React y components/ nunca hace fetch.** Los componentes reciben datos y emiten acciones; la red y el almacenamiento viven en hooks y servicios. Así la lógica del negocio se puede testear sin navegador (ya hay 14 pruebas).
- **Todo lo que cambia de un local a otro está en un solo archivo** (src/config/negocio.js): marca, colores, número de WhatsApp, envío, alias/CBU, textos y campos del checkout. Re-marcar la plantilla no requiere tocar componentes ni recompilar Tailwind (los colores se inyectan como variables CSS).

Reutilización por cliente (white-label en minutos)

Qué se cambia	Dónde	Efecto
Nombre, eslogan, logo	marca.*	Encabezado, pie, título de la página (SEO)
Paleta y redondeo	marca.tema / marca.radio	6 paletas listas: naranja, verde, azul, bordo, violeta, grafito

Qué se cambia	Dónde	Efecto
Número de WhatsApp	whatsapp.numero / nombreVendedor	Destino de todos los pedidos. Formato internacional: 549 + área + número
Origen de productos	catalogo.hojaCsv o hojaId+gid	Se conecta el Sheet del cliente
Envío	entrega.envio.* / entrega.retiro.*	Costo, envío gratis desde X, zona, demora, dirección y horarios
Formas de pago	pago.efectivo / pago.transferencia	Qué se ofrece al cliente y qué línea va al mensaje (acá se decide si hay que pasar el alias)
Identificación del pedido	pedido.mostrarNumeroOrden	Nombre y apellido del cliente por defecto; el número de pedido es opcional (apagado)
Formulario	pedido.campos.*	Qué se le pide al cliente (dirección sólo si hay envío)
Textos	textos.*	Pie legal, sin buscar en componentes

6. Seguridad y rendimiento

Superficie de ataque del sistema

Riesgo clásico	¿Aplica?	Por qué
Inyección SQL	No	No existe base de datos ni consultas: los datos son un CSV público de lectura.
Robo de credenciales	No	No hay login, ni tokens, ni API keys en el front. La hoja solo se lee.
Datos de tarjetas (PCI)	No	No se piden ni se procesan medios de pago en la web.
Datos bancarios publicados	No	La web no muestra alias ni CBU. Si el cliente paga por transferencia, el local comparte el alias por el chat, donde queda el pedido.
Rastro en el dispositivo del cliente	No	Sin localStorage, cookies ni sessionStorage: al recargar, no queda nada guardado en su teléfono.
Pagos fraudulentos	No	No hay pasarela: el cobro se acuerda por fuera y lo controla el local.
XSS desde la planilla	Mitigado	React escapa todo texto interpolado; no se usa innerHTML ni dangerouslySetInnerHTML.
Manipulación de precios	Mitigado	El total se recalcula desde los precios del catálogo; el mensaje lleva el detalle para que el local lo verifique.
Spam de pedidos	Bajo	No hay endpoint que atacar: el peor caso es un mensaje de WhatsApp, que el local puede bloquear.
Caída de Google Sheets	Mitigado	Caché en memoria del último menú: durante la visita se sigue mostrando lo ya cargado, sin escribir nada en el dispositivo.

Privacidad: dónde viven los datos del cliente

Dato	Dónde está	Dónde NO está
Nombre y apellido	En la memoria de la página mientras se arma el pedido y en el mensaje de WhatsApp que el cliente envía	No se transmite a ningún servidor, no va a la planilla, no se guarda en el dispositivo
Teléfono	Ídem: solo dentro del mensaje que el cliente manda	Sin base de datos, sin cookies, sin localStorage
Dirección, referencia y horario	Ídem (solo si elige envío a domicilio)	No se publica ni se comparte con terceros

No hay backend, ni base de datos, ni analítica, ni píxeles de redes sociales: la web no habla con nadie más que con el Google Sheet (solo lectura) y con los servidores de las fotos. Por eso no hay a quién filtrarle datos.

Qué está endurecido

- **Política de referencia *no-referrer*:** el navegador no manda la dirección de la página a otros sitios, así el link de WhatsApp (que lleva el pedido) no viaja como referencia.
- **Content-Security-Policy:** solo se ejecuta código y estilos propios, las imágenes van por https y las conexiones se limitan al Sheet configurado. Sin scripts de terceros.
- **Cabeceras de hosting** (vercel.json y public/_headers): X-Frame-Options DENY, frame-ancestors none (anti clickjacking), nosniff, Permissions-Policy (cámara, micrófono, ubicación y pagos bloqueados) y HSTS para forzar HTTPS.
- **Enlaces externos aislados:** todos con noopener noreferrer.
- **Sin formularios que envíen datos:** no hay ningún *form* que un bot pueda usar, así que no hay vía para inyectar spam desde la web.

- **Escapado de React:** nada de la planilla se inserta como HTML; si alguien escribe código en el Sheet, se muestra como texto.

Riesgos que quedan (para avisarlos, no para ocultarlos)

- **El número del local es público** (necesario para recibir pedidos): el spam puede llegar por WhatsApp, no por la web. WhatsApp tiene bloqueo y reporte.
- **WhatsApp/Meta procesa el mensaje** con los datos del cliente: queda bajo la responsabilidad del local no reenviarlos ni publicarlos y borrar pedidos viejos.
- **El link de WhatsApp queda en el historial del navegador del propio cliente** (su teléfono, su historial); con *no-referrer* esa dirección no se filtra a terceros.
- **Quien edite la planilla** puede cambiar precios y textos: cuenta de Google con verificación en dos pasos y hoja compartida solo con quien corresponda.
- **La hoja publicada como CSV es pública:** costos, márgenes y notas internas van en otra pestaña que no se publica.
- **Fotos en servidores de terceros:** ese servidor ve la IP del visitante. Conviene Drive, el hosting propio o un servicio de imágenes conocido.
- **Marco legal (Ley 25.326):** el criterio del proyecto es minimizar. La web no recolecta ni almacena datos; el pie de página ya declara que se usan solo para coordinar el pedido.

Rendimiento

- **Peso real medido:** 306 kB de JavaScript (95 kB gzip) y 37,6 kB de CSS (7,8 kB gzip). Una sola petición de datos: el CSV del Sheet (el resto queda en caché del navegador).
- **Sin dependencias pesadas:** solo React, Tailwind y PapaParse. Cero librerías de UI, de iconos o de estado (los iconos son SVG inline).
- **Carga diferida:** las 4 primeras imágenes del catálogo se cargan en modo *eager* con alta prioridad (mejora el LCP) y el resto en *lazy*.
- **Mobile-first:** una columna en celular, grilla de 2 a 4 columnas desde tablet/desktop; el carrito es un panel lateral y en el celular una barra fija inferior.
- **Accesibilidad:** foco visible, cierre con Escape, foco atrapado en el modal y en el panel, etiquetas asociadas a cada campo y respeto de "reducir movimiento".
- **Sin jerga de servidor:** es un paquete estático; se sirve desde CDN en cualquier hosting.

Cómo es el arranque en frío, paso a paso

```
npm install          # instala React, Vite, Tailwind y PapaParse
# 1. editar src/config/negocio.js (marca, whatsapp, envío, pago, hoja)
npm run validar public/productos-demo.csv # valida la planilla del cliente
npm run lint         # oxlint: 0 avisos esperados
npm test             # 21 pruebas de pedido, totales, parseo, mensaje y respaldos
npm run e2e          # flujo de compra completo en un navegador real (45 comprobaciones)
npm run privacidad   # audita red, datos del cliente y cabeceras (18 comprobaciones)
npm run dev          # desarrollo en http://localhost:5173
npm run build         # genera dist/ (paquete estatico listo para publicar)
npm run preview       # sirve dist/ localmente (puerto 4173)
npm run e2e          # flujo de compra completo en navegador: 35 comprobaciones
node scripts/capturar.mjs http://localhost:4173 docs/captura.png 1440 900 0 grilla
```

Publicación con hosting gratuito

- **Vercel:** `vercel deploy --prod` (framework Vite, salida dist). Ojo con la *Deployment Protection*: si está activa, el sitio responde 302 al login y no lo ve nadie. Se desactiva en Settings → Deployment Protection.
- **Netlify:** arrastrar la carpeta dist/ al panel, o `netlify deploy --prod --dir=dist` con un token personal.
- **Cloudflare Pages:** conectar el repo de GitHub desde el panel (`build npm run build`, salida dist).
- **GitHub Pages:** subir el contenido de dist a la rama gh-pages. El proyecto compila con rutas relativas, así que también funciona en subcarpetas de usuario. **Pero no admite cabeceras propias:** solo valen las declaradas en el HTML.
- **Dominio:** el negocio solo paga el dominio (por ejemplo .com.ar) y lo apunta al hosting. HTTPS queda incluido y es requisito para el portapapeles moderno (el botón Copiar tiene respaldo para contextos sin HTTPS).

Qué hosting conviene (precios verificados al escribir este documento)

	Plan gratis	Cabeceras propias	Uso comercial en el plan gratis	Plan pago más barato
Vercel	100 GB/mes de tráfico	Sí (vercel.json)	NO: Hobby es "solo uso personal no comercial"; el comercial exige Pro	Pro: US\$20/mes (por usuario, con US\$20 de crédito)
Netlify	Tráfico y builds por créditos	Sí (public/_headers)	Sí	Pro: US\$9/mes (1 miembro); US\$20/mes con miembros ilimitados
Cloudflare Pages	500 builds/mes, salida sin costo	Sí (public/_headers)	Sí	No hace falta para un sitio estático

Criterio para vender: la demo puede vivir en Vercel (Hobby) mientras sea vidriera. Cuando el local factura con el sitio, conviene **Netlify o Cloudflare Pages**, cuyo plan gratis permite uso comercial y aplica las mismas cabeceras; si necesita más, Netlify Pro cuesta la mitad que Vercel Pro.

7. Verificación realizada sobre este código

Todo lo que sigue se ejecutó sobre el código entregado, no es una descripción teórica.

Prueba	Resultado
Linters (oxlint)	OK — 0 avisos en src/ y scripts/
Compilación de producción	OK — 48 módulos, build en ~1 s
Pruebas de lógica (npm test)	OK — 21/21 (precios, totales, envío, validaciones, mensaje, link, respaldos de imagen, ingredientes y filtros)
Validador del menú (npm run validar)	OK — 18 productos, 4 categorías, 6 filtros rápidos, 0 problemas
Fotos del menú	41 URLs verificadas (HTTP 200); 18/18 cargan y se pintan (600×450)
E2E navegador real (npm run e2e)	OK — 45/45 comprobaciones, repetible entre corridas
Auditoría de privacidad (npm run privacidad)	OK — 18/18: red, datos del cliente, almacenamiento y cabeceras
Privacidad del dispositivo	OK — localStorage, sessionStorage y cookies en cero; el teléfono del cliente no viaja a ningún servidor
Ficha de producto	OK — abre desde la tarjeta, muestra ingredientes y agrega al pedido
Filtros rápidos	OK — los chips salen de la columna etiquetas (Vegetariano, Vegano...)
Respaldo de imágenes	OK — con un link roto: pide el primero, pide el segundo y muestra la foto
Bug de usabilidad (foco al escribir)	Corregido y verificado: el campo conserva el foco en cada tecla
Sin datos bancarios ni número de pedido	Verificado: ni la pantalla ni el mensaje los exponen
Responsive	OK — capturas en 1440×900 y 390×844 con barra de pedido fija en el celular

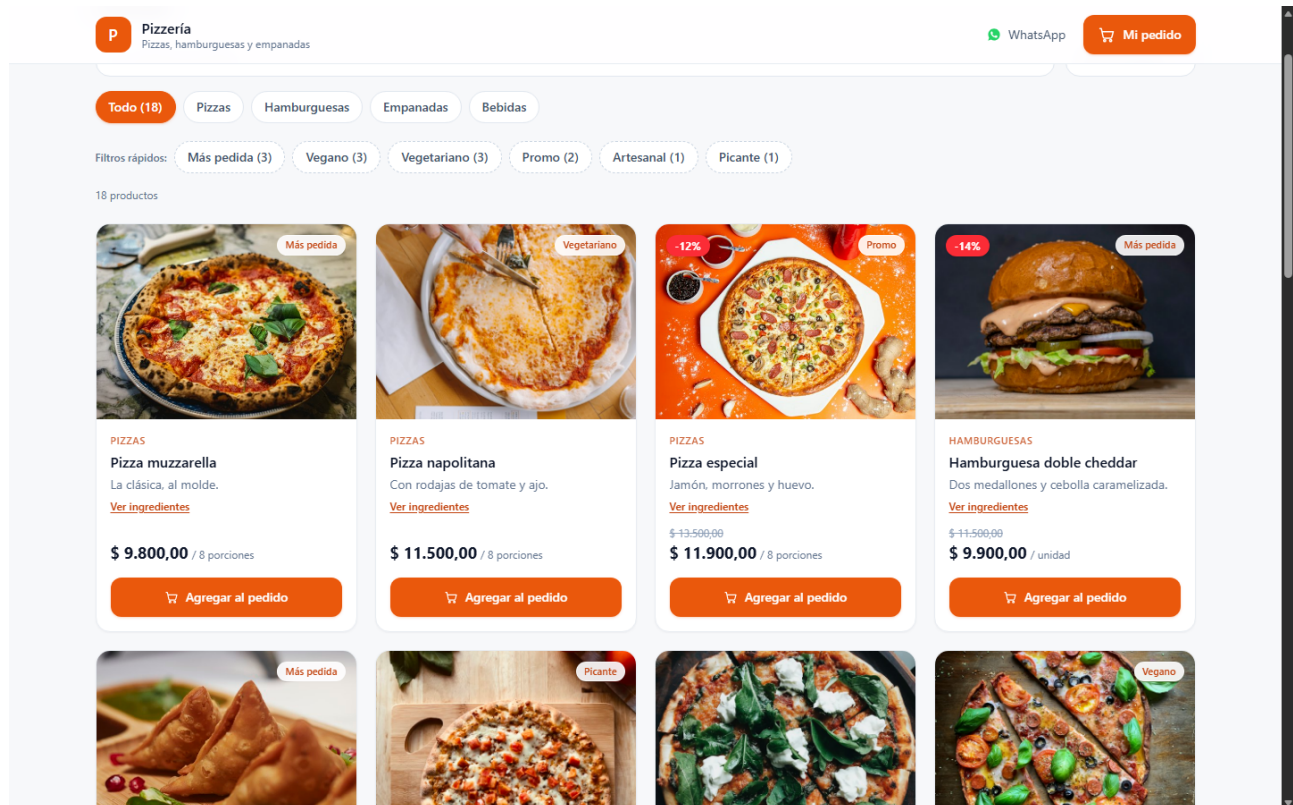
El E2E vive en `scripts/e2e.mjs` y escribe su informe en `docs/e2e-resultado.json`: recorre catálogo, búsqueda, filtros, carrito, checkout, transferencia, enlace de WhatsApp, pantalla de éxito y persistencia, y sale con código de error si alguna comprobación falla (se puede usar en CI).

Chequeos funcionales verificados en el navegador

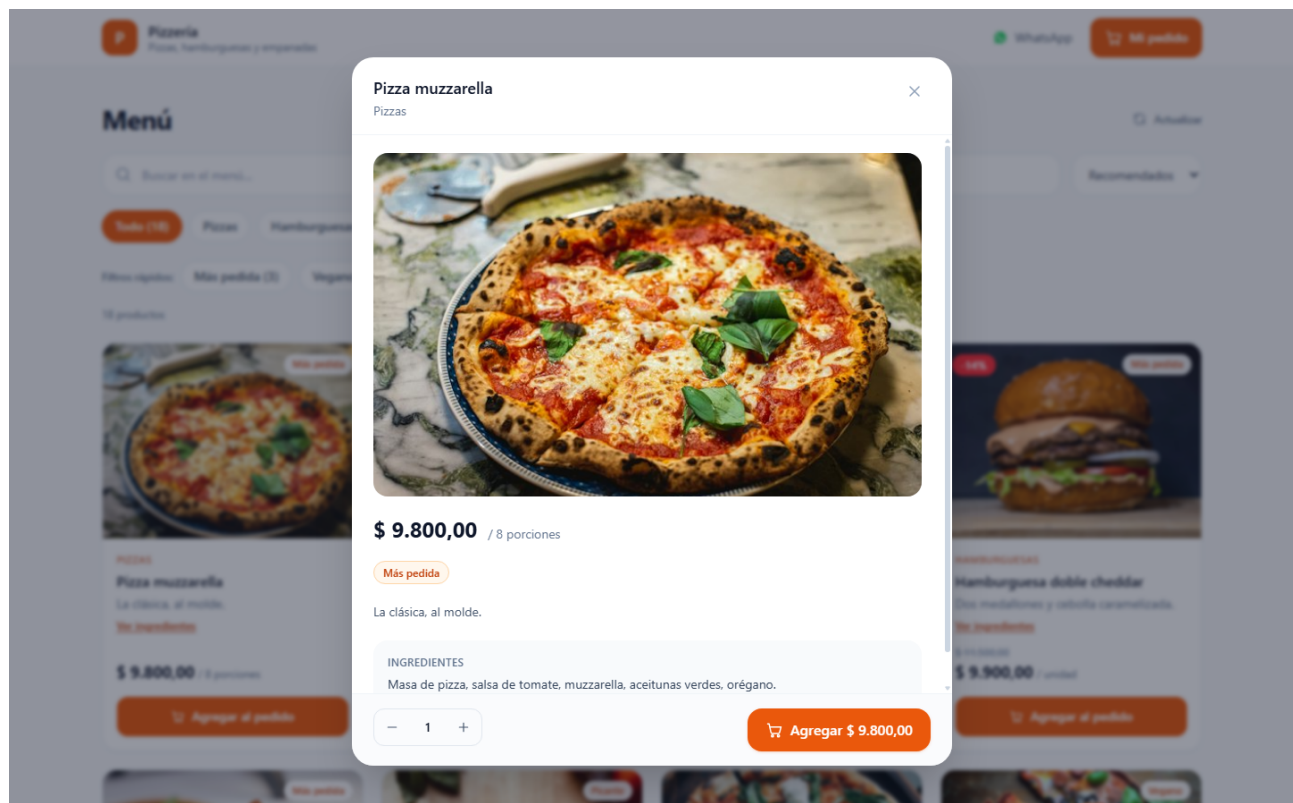
- El buscador filtra por título, descripción, categoría y etiquetas, sin distinguir tildes.
- Los filtros por categoría se generan solos desde la planilla (Rotisería, Panadería, Bebidas, Almacén, Verdulería en el ejemplo).
- Un producto con stock 0 se muestra como **Agotado** y no se puede agregar.
- Las ofertas muestran el precio anterior tachado y el porcentaje de descuento.
- El carrito suma cantidades del mismo producto, permite aclaraciones por línea ("sin cebolla") y calcula subtotal, envío y total.
- El envío gratis se aplica automáticamente al superar el monto configurado (y el carrito muestra cuánto falta para alcanzarlo).
- La validación del checkout marca cada campo con su error y no deja avanzar con datos incompletos.
- La pantalla de transferencia muestra Alias/CBU/titular/importe con botón Copiar (y avisa si el navegador bloquea el portapapeles).
- El paso final es un enlace real a `wa.me`: si el navegador bloquea ventanas emergentes, el usuario igual puede enviar el pedido.
- Al cerrar el flujo queda el número de orden visible y la opción de copiar el resumen.

Capturas del sistema funcionando

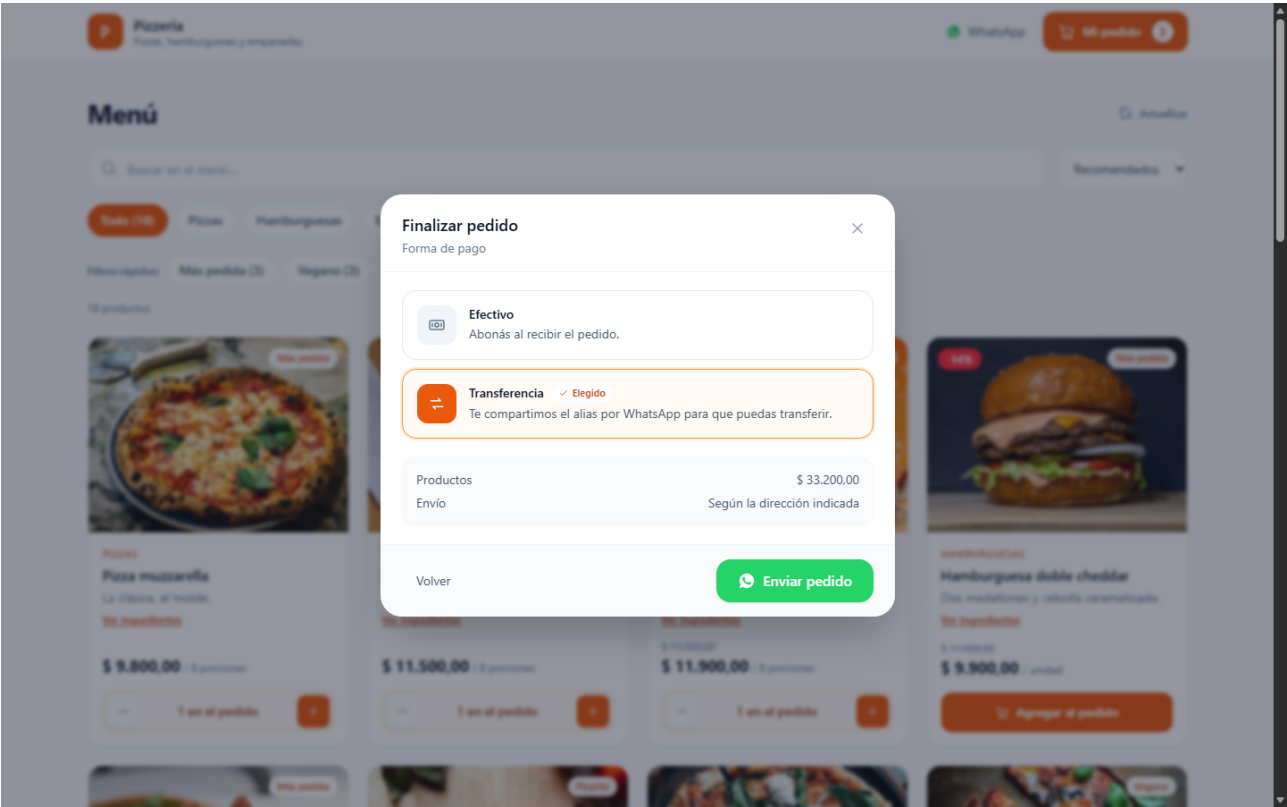
El menú es lo primero que ve el cliente (1440 px): categorías, filtros rápidos por etiqueta (Vegano, Vegetariano, Más pedida...), ofertas con precio tachado y el botón “Ver ingredientes” en cada tarjeta.



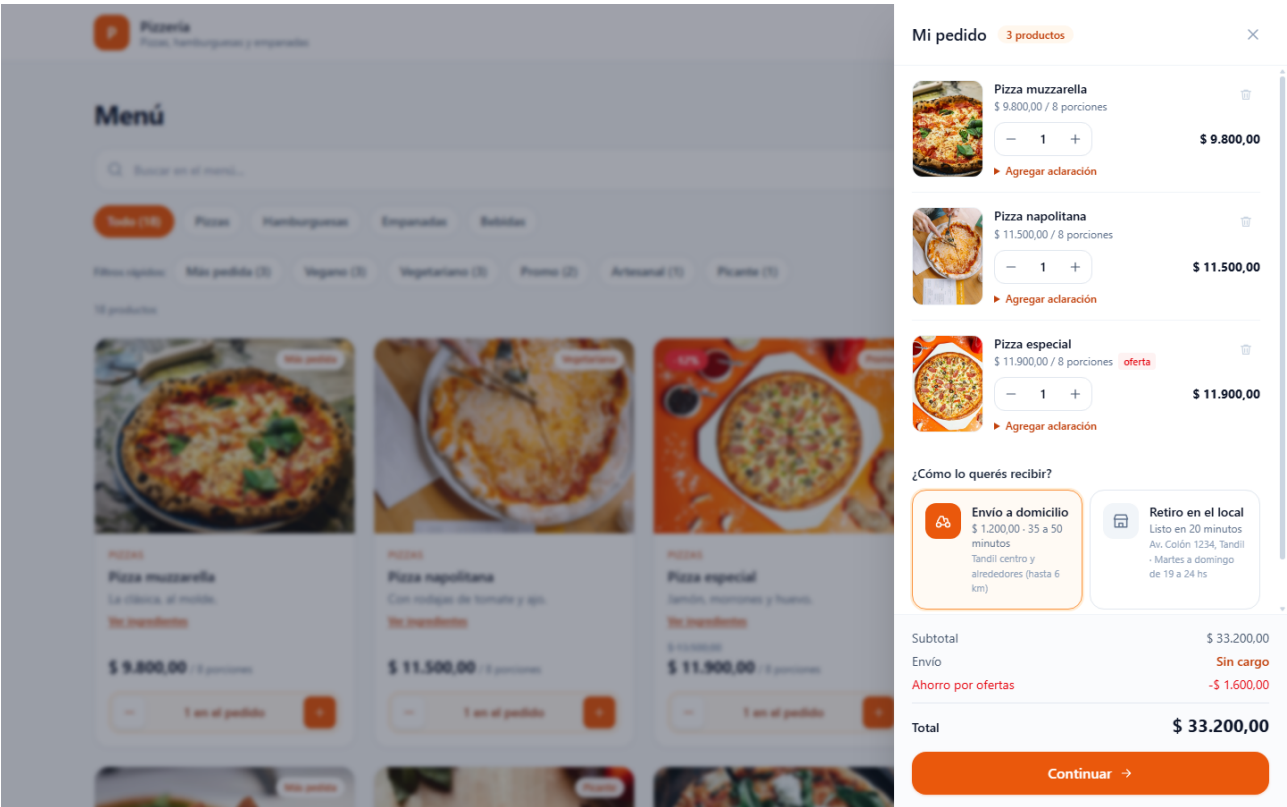
Ficha del producto: se abre desde la tarjeta y muestra foto grande, precio, etiquetas, descripción e ingredientes, con selector de cantidad. Resuelve el “¿qué lleva?” sin llamar al local.



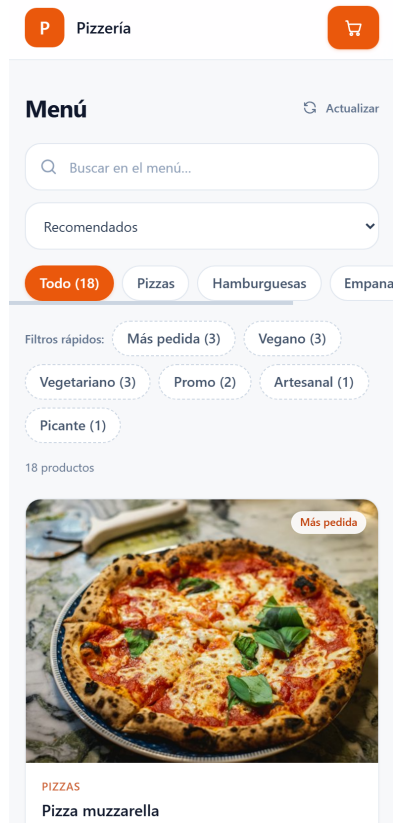
Pantalla de forma de pago: sólo se elige entre efectivo y transferencia. No hay datos bancarios, ni campos de tarjeta, ni un paso extra de confirmación: el botón verde abre WhatsApp con el pedido escrito.



Carrito lateral (off-canvas): cantidades, aclaración por producto, modalidad de entrega, barra de progreso hacia el envío gratis y totales.



Versión móvil (390 px): el menú arranca sin pantallas de presentación y la barra inferior fija mantiene el pedido a un toque de distancia.



Nota sobre las capturas: se tomaron con un navegador headless (sin ventana). En ese entorno las transiciones CSS no progresan y las imágenes con carga diferida no se descargan; el script `scripts/capturar.mjs` fuerza ambas cosas solo para poder fotografiar la página. El estado real se verificó midiendo el DOM (20/20 imágenes descargadas y visibles) y no confiando en la captura.

8. Límites conocidos y cómo venderlo como plantilla

Límites por diseño (conviene decirlos antes de vender)

- No hay stock en tiempo real ni reserva: el stock se actualiza a mano en la planilla y el local confirma por WhatsApp.
- El pedido vive en la memoria de la página: si el cliente recarga, arranca vacío (a cambio, no queda nada suyo guardado en el dispositivo).
- El menú se actualiza cuando el cliente recarga la página (la caché en memoria dura 10 minutos).
- La web no publica datos bancarios: el alias se pasa por el chat cuando el cliente elige transferencia (queda registrado en la conversación).
- El *id* de cada producto debe ser único y estable: es lo que identifica la línea del pedido.
- Los pedidos no quedan registrados en ningún sistema; el historial es el chat de WhatsApp.
- Para ver el modo demostración hay que servir el sitio (`dev/preview/hosting`): abrir el archivo con doble clic no permite leer el CSV por restricciones del navegador.
- Los precios son responsabilidad de la planilla: la web muestra exactamente lo que dice la hoja.

Checklist de entrega por local

- Cargar marca (nombre, eslogan, tema, logo) en `negocio.js`.
- Cargar el número de WhatsApp real y probar un pedido de punta a punta desde un celular.
- Configurar envío (costo, envío gratis desde, zona, demora) y datos de retiro.
- Definir con el dueño las formas de pago y la línea que va al mensaje (si hay que pasar el alias).
- Importar `docs/plantilla-google-sheet.csv` y cargar el menú real.
- Publicar la hoja como CSV, pegar la URL y correr el validador.
- Reemplazar el favicon y revisar los textos del pie.
- Publicar en el hosting y entregar la URL + una guía de una página para el dueño.

Qué se puede cobrar por esto

- **Como servicio:** implementación por cliente (marca + planilla + publicación + capacitación de 20 minutos). Es un trabajo de horas, no de semanas, porque la plantilla ya está hecha.
- **Como plantilla:** venta del código con licencia de uso por proyecto, con la guía de instalación y las 6 paletas incluidas.
- **Ingreso recurrente:** mantenimiento (nuevos productos, cambios de precios masivos, promociones) y hosting/dominio a nombre del cliente.

Resumen en una frase. Es una tienda que no necesita servidor: el catálogo vive en una planilla de Google, el carrito vive en el navegador del cliente y el pedido viaja por WhatsApp. El negocio solo paga el dominio y el desarrollo se entrega en horas, no en semanas.